NED University of Engineering & Technology

Department of Computer Science & Information Technology

SMART CAMPUS EVENT TICKETING & REAL-TIME EVENT PORTAL SYSTEM

Complex Computing Activity — Database Management System

DISCOVER Find and explore campus events effortlessly.

REGISTER Sign up and register for events easily.

SECURE Get instant, secure digital tickets.

REAL-TIME Live updates, seat counts & announcements.

Course	CT-261 — Database Management System (CLO-2, CLO-3)
Project Title	Smart Campus Event Ticketing & Real-Time Event Portal System
Submitted To	Miss Mehar Fatima & Miss Saba Izhar
Semester	Spring Semester 2026
Institution	NED University of Engineering & Technology
Degree	Bachelor of Science in Computer Science & IT

GROUP MEMBERS

Roll No.	Name
CT-24224	Muhammad Taha Farrukh
CT-24221	Muhammad Saad
CT-24227	UsaidUllah Qureshi
CT-24247	Muhammad Feras Malik

Table of Contents

Table of Contents 2
1. Problem Statement 5
2. Project Description 6
2.1 Project Overview 6
2.2 System Objectives 6
2.3 Technology Stack 6
3. Business Requirements 7
3.1 Functional Requirements 7
3.2 Non-Functional Requirements 7
3.3 Business Rules & Integrity Constraints 7
4. Current System Limitations 8
4.1 Overview of Existing System 8
4.2 Identified Limitations 8
4.3 Impact of Limitations 8
5. Team Participation 9
5.1 Meeting Log 9
6. Conceptual Design (ER Diagram) 10
6.1 ER Diagram 10
6.2 Entity Descriptions 11
6.3 Relationship Descriptions 12
7. Enhanced ER Design (EER Diagram) 13
7.1 EER Diagram 13
7.2 EER Concepts Applied 14
8. Functional Dependencies 15
9. Normalization Process 16
9.1 Un-Normalized Form (UNF) 16
Group A — Event & Organizer (multi-valued Organizer columns) 16
Group B — Competition (multi-valued columns per Event row) 16
Group C — Team & TeamMember (multi-valued per Competition row) 16
Group D — Registration, Payment & Ticket (multi-valued per Team row) 17
9.2 Anomalies in UNF 17
Insertion Anomaly 17
Deletion Anomaly 18
Updation Anomaly 18
9.3 First Normal Form (1NF) 19
9.4 Second Normal Form (2NF) 20
2NF — Users Table 20

2NF — Event Table 20
2NF — Competition Table 21
2NF — Team Table 21
2NF — TeamMember Table 21
9.5 Third Normal Form (3NF) 22
3NF — Users (base table; role-specific columns removed) 22
3NF — Student, Organizer, Admin, Volunteer Sub-Tables 22
3NF — Event Table (CreatedBy_Name/Email replaced with FK) 24
3NF — Team Table (LeaderName/Email replaced with LeaderUserID FK) 24
3NF — TeamMember Table 24
3NF — Registration Table (UserName → UserID FK) 24
3NF — Payment Table (VerifierName → VerifiedBy FK) 25
3NF — Ticket Table 25
9.6 Boyce-Codd Normal Form (BCNF) 26
BCNF Violation Example 26
BCNF Fix 26
10. SQL Queries 27
10.1 DDL — Table Creation 27
10.2 DML — Data Insertion 29
10.3 SELECT Queries 30
Query 1: All Events with Organizer Details 30
Query 2: Registration Status with Payment & Ticket 30
Query 3: Volunteer Count per Event 30
Query 4: Teams with Member Count per Competition 30
Query 5: Pending Payments Awaiting Verification 30
Query 6: Students Without Any Registration 31
10.4 UPDATE and DELETE Queries 31
10.5 Views, Triggers, and Stored Procedures 31
Trigger 1: Auto-generate Ticket on Payment Approval 31
Trigger 2: Enforce MaxTeamSize 32
Trigger 3: Payment Verified Only by Organizer or Volunteer 32
11. Screenshots / Query Outputs 33
11.1 Database Schema in Supabase 33
11.2 Query 1 Output — Events with Organizer Details 34
11.3 Query 2 Output — Registration Status with Payment & Ticket 34
11.4 Trigger Demonstration — Auto Ticket Generation 35
11.5 Additional Outputs 35
Team Member Count per Competition 35
Students Without Registration 36

Total Revenue per Event 37
Approved Tickets List 37
12. Group Member Contributions 38
13. Limitations 38
13.1 Technical Limitations 38
13.2 Design Limitations 38
13.3 Time & Resource Constraints 38
14. Future Enhancements 39
14.1 Short-Term Enhancements 39
14.2 Long-Term Enhancements 39
15. Conclusion 40

1. Problem Statement

Current university campus systems suffer from fragmented, informal approaches to managing events. The following issues were identified through stakeholder analysis at NED University of Engineering & Technology:

- No centralized database for event discovery, registration, or tracking.
- Manual, error-prone payment verification via WhatsApp screenshots.
- Inability to prevent duplicate registrations or enforce team size limits programmatically.
- Loss of event data due to reliance on fragmented communication channels (social media, notice boards, chat groups).
- No digital audit trail for tickets, payments, or organizer approvals.
- Organizer and volunteer role management is entirely manual and informal.

These issues result in administrative overhead, data inconsistency, and a poor experience for students and faculty alike. A structured, normalized relational database system is required to replace these ad-hoc processes.

2. Project Description

2.1 Project Overview

The Smart Campus Event Ticketing & Real-Time Event Portal System is a centralized digital platform designed to manage all university events efficiently. It replaces the scattered approach of sharing event information over WhatsApp, social media, and physical notice boards with a single structured portal.

The system supports four distinct user roles: Students, Organizers, Volunteers, and Admins each with specific access rights and capabilities enforced at the database level through role-based constraints.

2.2 System Objectives

- Provide a single, authoritative platform for all campus event information and registration.
- Enforce business rules (team size limits, duplicate prevention, role validation) at the database layer.
- Automate the ticket generation workflow: Registration → Payment Submission → Verification → QR Ticket.
- Maintain a full digital audit trail of registrations, payments, verifications, and approvals.
- Support role-based access with specialization (a student can also be an organizer).

2.3 Technology Stack

Component	Technology
Backend	ASP.NET Core (.NET 8, C#)
Frontend	ASP.NET MVC / Razor Pages, Bootstrap, JavaScript
Database	Supabase (PostgreSQL)
Authentication	Role-based via Users table & Supabase RLS

3. Business Requirements

3.1 Functional Requirements

- Users can register as Students; Admins can promote users to Organizer or Volunteer roles.
- Organizers can create and manage Events and Competitions linked to those events.
- Students can register for competitions (individually or as a team).
- Each registration must be followed by a payment screenshot upload.
- Organizers or Volunteers can approve or reject payment submissions.
- Volunteers can be assigned to specific events or competitions with designated roles.
- Admin can view all system data and manage role requests.

3.2 Non-Functional Requirements

- Data Integrity: All constraints (UNIQUE, NOT NULL, CHECK, FK) enforced at the database level.
- Scalability: Schema designed for horizontal scalability with PostgreSQL (Supabase).
- Security: Role-level access control via Supabase Row Level Security (RLS) policies.
- Availability: Cloud-hosted database ensures high uptime.
- Performance: Indexes on all FK and frequently queried columns.

3.3 Business Rules & Integrity Constraints

#	Rule	Enforcement Mechanism
1	No duplicate registration per competition	UNIQUE(UserID, CompetitionID) on Registration
2	Team size ≤ MaxTeamSize	Trigger: trg_team_size_limit on TeamMember INSERT
3	Ticket generated only after payment approval	Trigger: trg_auto_ticket on Payment UPDATE
4	Payment verified only by Organizer/Volunteer	Trigger: trg_payment_verifier
5	Registration type matches MaxTeamSize	Trigger: trg_registration_type
6	Competition dates within Event dates	Trigger: trg_competition_dates
7	Student RollNumber is unique	UNIQUE constraint on Student.RollNumber
8	Email must be unique system-wide	UNIQUE constraint on Users.Email
9	Payment status syncs to Registration	Trigger: trg_payment_status_sync
10	Organizer cannot be deleted with active events	ON DELETE RESTRICT on Event.CreatedBy

4. Current System Limitations

4.1 Overview of Existing System

Prior to this system, all event management at the university was handled informally. Organizers would post event details on WhatsApp groups, social media pages, and physical notice boards. Students registered by messaging an organizer directly or filling google/paper forms. Payments were made via bank transfer with screenshots shared on chat. Tickets, if issued at all, were verbal confirmations or PDF files sent manually.

4.2 Identified Limitations

- Data Redundancy: Event and organizer information was repeated across multiple spreadsheets and messages with no single source of truth.
- No Duplicate Prevention: A student could register for the same competition multiple times from different accounts.
- Manual Payment Verification: Screenshots were matched against bank statements by hand, leading to errors and delays.
- No Audit Trail: Deleted WhatsApp messages or lost files could permanently destroy registration history.
- No Team Size Enforcement: Organizers had to manually count team members, leading to over-sized teams.
- No Role Management: Any person could claim to be a volunteer or organizer with no system verification.
- Fragmented Data: Event data lived across Google Forms, WhatsApp chats, and local Excel files — none integrated.

4.3 Impact of Limitations

These limitations caused significant issues: students were unable to verify their registration status, payments were occasionally lost or duplicated, and event organizers had no reliable way to generate attendance lists or enforce competition rules. The absence of a centralized system also made it impossible to produce analytical reports on participation trends or volunteer deployment.

5. Team Participation

Roll No.	Member Name	Contribution / Tasks Completed
CT-24224	Muhammad Taha Farrukh	Backend & Database Integration, bug fixing
CT-24221	Muhammad Saad	Identified Anomalies & Performed Normalization
CT-24227	UsaidUllah Qureshi	Frontend Enhancement
CT-24247	Muhammad Feras Malik	Database Design & Implementation

5.1 Meeting Log

Meeting #	Date	Agenda / Decisions Made
1	24th Feb	Project topic selected, roles assigned, entity identification started
2	10th Mar	ER diagram reviewed and finalized, EER concepts confirmed
3	20th Mar	Normalization steps reviewed, SQL schema drafted
4	10th Apr	Triggers and business rules added, sample queries tested
5	23rd Apr	Final review, report compilation and submission preparation

6. Conceptual Design (ER Diagram)

6.1 ER Diagram

.

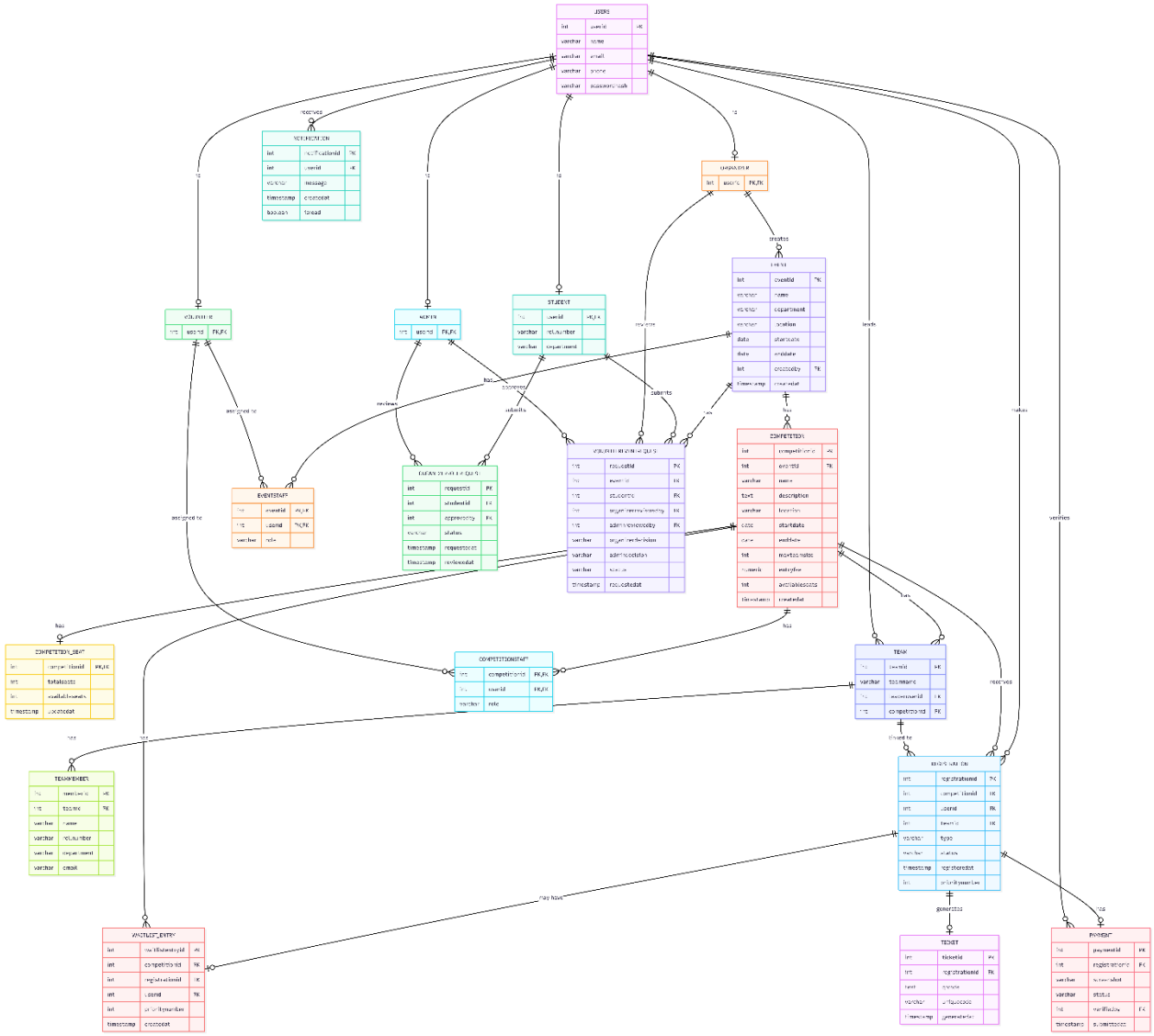


Figure 6.1 — Entity-Relationship Diagram: Smart Campus Event Ticketing & Real-Time Event Portal System

6.2 Entity Descriptions

Entity	Primary Key	Key Attributes
USER (super-entity)	UserID	Name, Email (UNIQUE), Phone
STUDENT	UserID (PK, FK)	RollNumber (UNIQUE), Department
ORGANIZER	UserID (PK, FK)	Inherits all User attributes
VOLUNTEER	UserID (PK, FK)	Inherits all User attributes
ADMIN	UserID (PK, FK)	Full system access
EVENT	EventID	Name, Department, Location, StartDate, EndDate, CreatedBy (FK)
COMPETITION	CompetitionID	EventID (FK), Name, Description, Location, StartDate, EndDate, MaxTeamSize, EntryFee, `AvailableSeats, CreatedAt`
TEAM	TeamID	TeamName, LeaderUserID (FK), CompetitionID (FK)
TEAM_MEMBER	MemberID	TeamID (FK), Name, RollNumber, Department, Email
REGISTRATION	RegistrationID	CompetitionID (FK), UserID (FK), TeamID (FK), Type, Status, RegisteredAt, PriorityNumber
PAYMENT	PaymentID	RegistrationID (UNIQUE FK), Screenshot, Status, VerifiedBy (FK)
TICKET	TicketID	RegistrationID (UNIQUE FK), QRCode (UUID), UniqueCode
COMPETITION_SEAT	CompetitionID (PK, FK)	TotalSeats, AvailableSeats, UpdatedAt (1-to-1 extension table of COMPETITION)
WAITLIST_ENTRY	WaitlistEntryID	CompetitionID (FK), RegistrationID (UNIQUE FK), UserID (FK), PriorityNumber, CreatedAt
NOTIFICATION	NotificationID	UserID (FK), Message, IsRead, CreatedAt
ORGANIZERROLEREQUEST	RequestID	StudentID (FK), ApprovedBy (FK→Admin), Status (Pending/Approved/Rejected), RequestedAt, ReviewedAt
VOLUNTEEREVENTREQUEST	RequestID	EventID (FK), StudentID (FK), OrganizerReviewedBy (FK→Organizer), AdminReviewedBy (FK→Admin), OrganizerDecision, AdminDecision, Status, RequestedAt
EVENTSTAFF	EventID (PK, FK), UserID (PK, FK)	Role
COMPETITIONSTAFF	CompetitionID (PK, FK), UserID (PK, FK)	Role

6.3 Relationship Descriptions

Relationship	Entities Involved	Cardinality	Enforcement
is	USER → STUDENT / ORGANIZER/VOLUNTEER/ADMIN	1:1	Shared PK+FK (EER Specialization)
creates	ORGANIZER → EVENT	1:M	EventID FK, RESTRICT on delete
contains	EVENT → COMPETITION	1:M	EventID FK on Competition
has	COMPETITION → TEAM	1:M	CompetitionID FK on Team
includes	TEAM → TEAM_MEMBER	1:M	TeamID FK on TeamMember
registers	USER → REGISTRATION	1:M	UserID FK on Registration
for	COMPETITION → REGISTRATION	1:M	CompetitionID FK on Registration
has	REGISTRATION → PAYMENT	1:1	UNIQUE RegistrationID on Payment
generates	REGISTRATION → TICKET	1:1	UNIQUE RegistrationID on Ticket
assigned_to (Event)	VOLUNTEER ↔ EVENT	M:N	EventStaff junction table
assigned_to (Comp)	VOLUNTEER ↔ COMPETITION	M:N	CompetitionStaff junction table
verifies	USER → PAYMENT (VerifiedBy FK; role enforced by trigger)	1:M	VerifiedBy FK + trigger check
tracks seats	COMPETITION → COMPETITION_SEAT	1:1	CompetitionID FK on Competition_Seat (Shared PK+FK (1-to-1 extension))
queued in	REGISTRATION → WAITLIST_ENTRY	1:1	UNIQUE RegistrationID on Waitlist_Entry
notified	USER → NOTIFICATION	1:M	UserID FK on Notification
requests role	STUDENT → ORGANIZERROLEREQUEST	1:M	StudentID FK on OrganizerRoleRequest
approves request	ADMIN → ORGANIZERROLEREQUEST	1:M	ApprovedBy FK on OrganizerRoleRequest
leads	USER → TEAM	1:M	LeaderUserID FK on Team
linked to	TEAM → REGISTRATION	1:M	TeamID FK on Registration
Has (VolEvent)	EVENT → VOLUNTEEREVENTREQUEST	1:M	EventID FK on VolunteerEventRequest
submits (VolEvent)	STUDENT → VOLUNTEEREVENTREQUEST	1:M	StudentID FK on VolunteerEventRequest
reviews (VolEvent)	ORGANIZER → VOLUNTEEREVENTREQUEST	1:M	OrganizerReviewedBy FK on VolunteerEventRequest
approves (VolEvent)	ADMIN → VOLUNTEEREVENTREQUEST	1:M	AdminReviewedBy FK on VolunteerEventRequest

7. Enhanced ER Design (EER Diagram)

7.1 EER Diagram

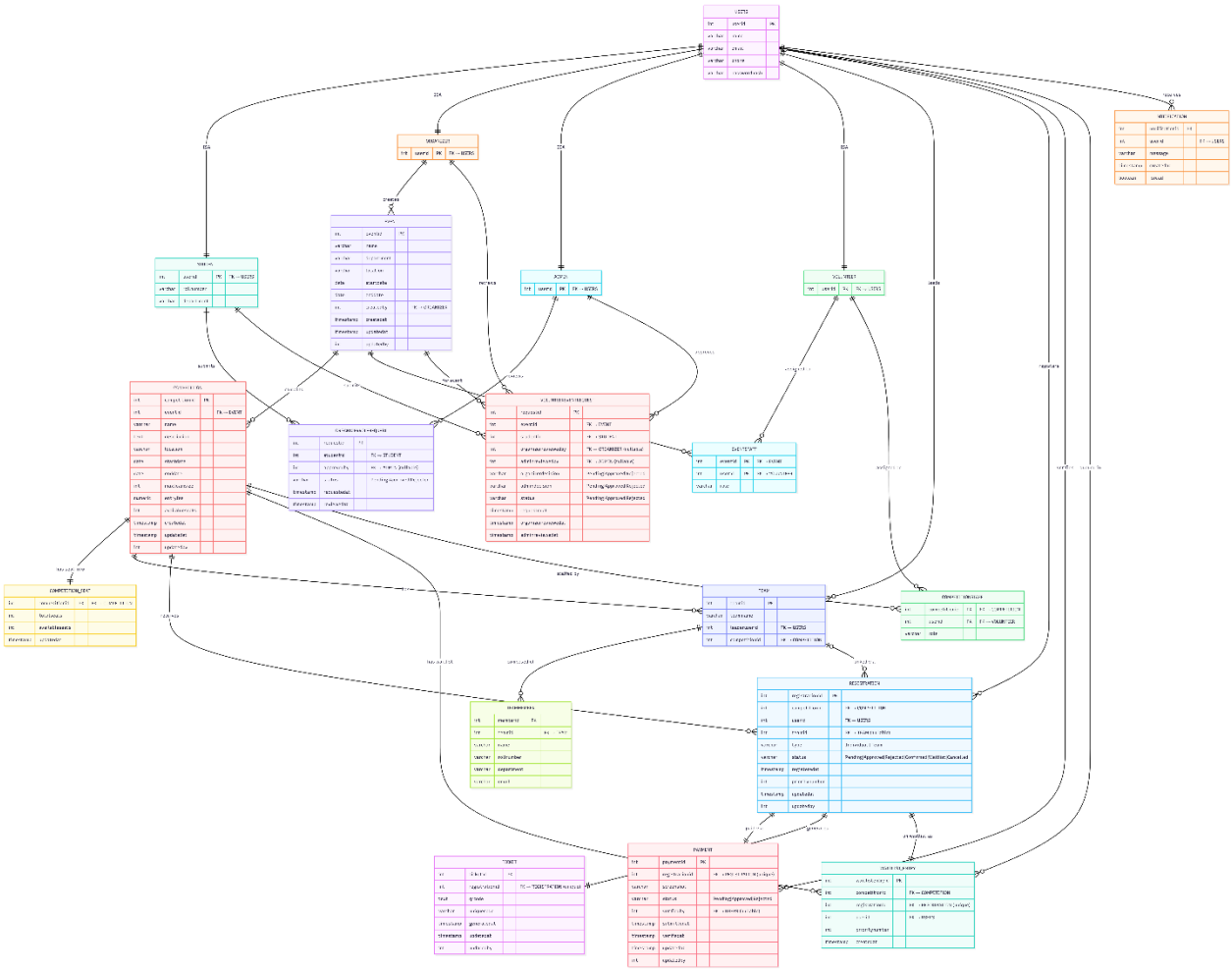


Figure 7.1 — Enhanced ER Diagram: Specialization hierarchy for USER super-entity

7.2 EER Concepts Applied

EER Specialization Summary

- USER is the super-entity (base class).
- Sub-entities: STUDENT, ORGANIZER, VOLUNTEER, ADMIN.
- Type: OVERLAPPING — a user can simultaneously hold multiple roles (e.g., a Student can also become an Organizer). Each active role has a row in its respective sub-table sharing the same UserID.
- Participation: PARTIAL — not every user must belong to a subtype.
- Implementation: Each sub-entity shares the UserID primary key with the parent (table-per-hierarchy pattern).

The overlapping constraint means a student who applies for and receives organizer privileges will have a row in both the Student and Organizer sub-tables, linked by the same UserID in the parent Users table. The partial constraint means volunteer/admin accounts exist without being students — such as external staff members.

8. Functional Dependencies

The following functional dependencies (FDs) hold in the final BCNF-normalized schema. For every table, the primary key (or candidate key) is the sole determinant of all other attributes — confirming BCNF compliance.

Table	Functional Dependencies
Users	UserID → Name, Email, Phone
Student	UserID → RollNumber, Department
Organizer / Volunteer / Admin	UserID → (all attributes inherited from Users)
Event	EventID → Name, Department, Location, StartDate, EndDate, CreatedBy
Competition	CompetitionID → EventID, Name, Description, Location, StartDate, EndDate, MaxTeamSize, EntryFee
EventStaff	(EventID, UserID) → Role
CompetitionStaff	(CompetitionID, UserID) → Role
Team	TeamID → TeamName, LeaderUserID, CompetitionID Also: (TeamName, CompetitionID) → TeamID [candidate key]
TeamMember	MemberID → TeamID, Name, RollNumber, Department, Email Also: (TeamID, Email) → MemberID [candidate key — BCNF fix]
Registration	RegistrationID → CompetitionID, UserID, TeamID, Type, Status, RegisteredAt Also: (UserID, CompetitionID) → RegistrationID [candidate key]
Payment	PaymentID → RegistrationID, Screenshot, Status, VerifiedBy, SubmittedAt, VerifiedAt Also: RegistrationID → PaymentID [1:1 — both candidate keys]
Ticket	TicketID → RegistrationID, QRCode, UniqueCode, GeneratedAt Also: RegistrationID → TicketID, QRCode → TicketID [all candidate keys]
Competition_Seat	CompetitionID → TotalSeats, AvailableSeats, UpdatedAt (CompetitionID is sole superkey; strong entity with 1:1 FK to Competition — has its own PK)
Waitlist_Entry	WaitlistEntryID → CompetitionID, RegistrationID, UserID, PriorityNumber, CreatedAt Also: RegistrationID → WaitlistEntryID [candidate key — 1:1 with Registration]
Notification	NotificationID → UserID, Message, IsRead, CreatedAt
OrganizerRoleRequest	RequestID → StudentID, ApprovedBy, Status, RequestedAt, ReviewedAt
VolunteerEventRequest	RequestID → EventID, StudentID, OrganizerReviewedBy, AdminReviewedBy, OrganizerDecision, AdminDecision, Status, RequestedAt Also: (EventID, StudentID) → RequestID [candidate key]

9. Normalization Process

Normalization is the process of organizing a database to reduce redundancy and improve data integrity. It is applied in progressive steps: 1NF removes repeating groups; 2NF eliminates partial dependencies; 3NF removes transitive dependencies; BCNF ensures every determinant is a candidate key.

9.1 Un-Normalized Form (UNF)

The single flat table stores all system data together. It violates 1NF because the following columns contain multiple values in a single cell:

- Organizer_Names, Organizer_Emails
- Competition_Names, Competition_Descriptions, Competition_Locations, Competition_StartDates, Competition_EndDates, Competition_MaxTeamSizes, Competition_EntryFees
- Team_Names, TeamMember_Names, TeamMember_RollNumbers, TeamMember_Departments, TeamMember_Emails
- Registration_Types, Registration_Statuses, Payment_Screenshots, Pay_Statuses, Pay_VerifiedBy, Ticket_QRCodes, Ticket_UniqueCodes

Group A — Event & Organizer (multi-valued Organizer columns)

Event_Name	Event_Dept	Event_Location	Ev_StartDate	Ev_EndDate	Organizer_Names	Organizer_Emails
TechFest 2026	CSIT	Department	2026-01-05	2026-01-06	Saad Ali; Muhammad Taha Farrukh	saadali@ned.com ; mtahafarrukh@gmail.com

Group B — Competition (multi-valued columns per Event row)

Competition_Names	Comp_Locations	Comp_StartDates	Comp_EndDates	Comp_MaxTeamSizes	Comp_EntryFees
Speed Programming	lab 2	2026-01-05	2026-01-05	2	1000.00

Group C — Team & TeamMember (multi-valued per Competition row)

Team_Names	TeamMember_Names	TM_RollNumbers	TM_Departments	TM_Emails
Muhammad Taha Farrukh;	Muhammad Taha Farrukh; Feras Malik; Feras Malik;	CT-24224; CT-24247; CT-	CSIT; CSIT; CSIT; CSIT	mtahafarrukh@gmail.com; ferasmalik@ned.com;

Team_Names	TeamMember_Names	TM_RollNumbers	TM_Departments	TM_Emails
Main Akela he kaafi hun	Muhammad Taha Farrukh	24247; CT-24224		ferasmalik@ned.com; mtahafarrukh@gmail.com

Group D — Registration, Payment & Ticket (multi-valued per Team row)

Reg_Types	Reg_Statuses	Payment_Screenshots	Pay_Statuses	Pay_VerifiedBy	Ticket_UniqueCodes
Team; Team	Approved; Approved	d4fb7e03....jpeg; 010f2187....jpeg	Approved; Approved	Saad Ali; Uzair Ahmed	92C4E9BB; 198935D4

9.2 Anomalies in UNF

Insertion Anomaly

An insertion anomaly occurs when a valid piece of information cannot be stored without adding unrelated data. In the flat table, a new organizer or volunteer cannot be recorded unless they are already linked to an event, competition, team, payment, and ticket.

Example: Organizer "Dr. Nadia" (nadia@ned.edu) has just been granted organizer access but has not yet created any event. Row 3 below cannot be inserted because all non-organizer columns are empty — this is the Insertion Anomaly.

Event_Name	Competition_Name	Organizer_Name	Organizer_Email	Team_Name	Ticket_UniqueCode
TechFest 2026	Speed Programming	Saad Ali	saadali@ned.com	Muhammad Taha Farrukh	92C4E9BB
TechFest 2026	Speed Programming	Saad Ali	saadali@ned.com	Main Akela he kaafi hun	198935D4
— (no event yet)	—	Dr. Nadia	nadia@ned.edu	—	—

Deletion Anomaly

A deletion anomaly occurs when deleting a record unintentionally destroys other valuable data.

Example: If Team "Main Akela he kaafi hun" (the only team in Registration ID 3) is cancelled and its row is deleted, the associated Payment screenshot and QR-coded Ticket (UniqueCode: 198935D4) are permanently lost, even though the competition itself still exists.

Reg_ID	Team_Name	Competition	Payment_Status	VerifiedBy	Ticket_UniqueCode
2	Muhammad Taha Farrukh	Speed Programming	Approved	Saad Ali	92C4E9BB
3	Main Akela he kaafi hun	Speed Programming	Approved	Uzair Ahmed	198935D4

Updation Anomaly

An update anomaly occurs when redundant data requires the same change to be made in multiple rows. Missing even one occurrence leaves the database inconsistent.

Example: If the venue of TechFest 2026 changes from "Department" to "Main Auditorium", every row in the unnormalized table that references TechFest 2026 must be updated. Since team member data causes the event to repeat for each member, missing even one row creates conflicting location values.

Event_Name	Event_Location	Competition_Name	TeamMember_Name	TeamMember_Email
TechFest 2026	Department	Speed Programming	Muhammad Taha Farrukh	mtahafarrukh@gmail.com
TechFest 2026	Department	Speed Programming	Feras Malik	ferasmalik@ned.com
TechFest 2026	Main Auditorium	Speed Programming	Feras Malik	ferasmalik@ned.com
TechFest 2026	Department	Speed Programming	Muhammad Taha Farrukh	mtahafarrukh@gmail.com

9.3 First Normal Form (1NF)

First Normal Form requires each column to hold exactly one atomic value. All multi-valued (semicolon-separated) cells are split into individual rows.

1NF Fix: All multi-valued columns (semicolon-separated lists) are split into individual rows. Candidate Key: [Event_Name, Organizer_Name, Competition_Name, Team_Name, Member_RollNo, Ticket_Code]

1NF Table — Representative Rows (each row now holds one atomic value per cell)

Event_Name	Organizer_Name	Competition_Name	Team_Name	Member_Name	Member_RollNo	Reg_Type	Pay_Status	Ticket_Code
TechFest 2026	Saad Ali	Speed Programming	Muhammad Taha Farrukh	Muhammad Taha Farrukh	CT-24224	Team	Approved	92C4E9BB
TechFest 2026	Saad Ali	Speed Programming	Muhammad Taha Farrukh	Feras Malik	CT-24247	Team	Approved	92C4E9BB
TechFest 2026	Saad Ali	Speed Programming	Main Akela he kaafi hun	Feras Malik	CT-24247	Team	Approved	198935D4
TechFest 2026	Saad Ali	Speed Programming	Main Akela he kaafi hun	Muhammad Taha Farrukh	CT-24224	Team	Approved	198935D4
TechFest 2026	Muhammad Taha Farrukh	Speed Programming	Muhammad Taha Farrukh	Muhammad Taha Farrukh	CT-24224	Team	Approved	92C4E9BB
TechFest 2026	Muhammad Taha Farrukh	Speed Programming	Muhammad Taha Farrukh	Feras Malik	CT-24247	Team	Approved	92C4E9BB
TechFest 2026	Muhammad Taha Farrukh	Speed Programming	Main Akela he kaafi hun	Feras Malik	CT-24247	Team	Approved	198935D4
TechFest 2026	Muhammad Taha Farrukh	Speed Programming	Main Akela he kaafi hun	Muhammad Taha Farrukh	CT-24224	Team	Approved	198935D4

Observation: The 1NF table achieves atomic values per cell. However, massive redundancy remains: Event_Name, Competition_Name, Pay_Status repeat identically across rows. Many non-key attributes depend only on part of the composite candidate key, a partial dependency addressed in 2NF.

9.4 Second Normal Form (2NF)

Second Normal Form requires that every non-key attribute depends on the entire candidate key, not just part of it. Partial dependencies are identified and resolved by decomposition.

- Event_Location, Event_StartDate, Event_EndDate → depend only on Event_Name
- Competition attributes → depend only on Competition_Name
- TeamMember details (Name, Department, Email) → depend only on Member_RollNo
- Team attributes → depend only on Team_Name
- Payment & Ticket data → depend only on the Registration, not the full composite key

2NF — Users Table

Role-specific columns still present at this stage — the transitive dependency (Role → RollNumber/Department) is resolved in 3NF.

User ID	Name	Email	Phone	Role	RollNumber	Department	Password Hash
1	Muhammad Taha Farrukh	mtahafarrukh@gmail.com	03332758016	Student	CT-24224	CSIT	$2a$11$RZi...
2	Feras Malik	ferasmalik@ned.com	03331234567	Student	CT-24247	CSIT	$2a$11$qFc...
4	Saad Ali	saadali@ned.com	03332758017	Organizer	—	—	$2a$11$zM1...
5	UsaidUllah Qureshi	usaid@ned.com	03121231231	Admin	—	—	$2a$11$Lpb...
6	Uzair Ahmed	uzair@ned.com	03332758018	Volunteer	—	—	$2a$11$ayQ...

2NF — Event Table

CreatedBy_Name → CreatedBy_Email is still a transitive dependency at this stage — resolved in 3NF.

EventID	Name	Department	Location	StartDate	EndDate	CreatedBy_Name	CreatedBy_Email
1	TechFest 2026	CSIT	Department	2026-01-05	2026-01-06	Saad Ali	saadali@ned.com

2NF — Competition Table

CmplID	EventID	Name	Description	Location	StartDate	EndDate	MaxTeamSize	EntryFee
2	1	Speed Programming	Competitive coding challenge	lab 2	2026-01-05	2026-01-05	2	1000.00

2NF — Team Table

LeaderName → LeaderEmail is still a transitive dependency at this stage — resolved in 3NF.

TeamID	TeamName	LeaderName	LeaderEmail	CompetitionID
1	Muhammad Taha Farrukh	Muhammad Taha Farrukh	mtahafarrukh@gmail.com	2
2	Main Akela he kaafi hun	Feras Malik	ferasmalik@ned.com	2

2NF — TeamMember Table

MemberID	TeamID	Name	RollNumber	Department	Email
1	1	Muhammad Taha Farrukh	CT-24224	CSIT	mtahafarrukh@gmail.com
2	1	Feras Malik	CT-24247	CSIT	ferasmalik@ned.com
3	2	Feras Malik	CT-24247	CSIT	ferasmalik@ned.com
4	2	Muhammad Taha Farrukh	CT-24224	CSIT	mtahafarrukh@gmail.com

9.5 Third Normal Form (3NF)

Third Normal Form requires that no non-key attribute is transitively dependent on the primary key. The following transitive dependencies from 2NF are resolved:

- Users: UserID → Role → (RollNumber, Department) — resolved by splitting into sub-tables: Student, Organizer, Admin, Volunteer.
- Event: EventID → CreatedBy_Name → CreatedBy_Email — resolved by replacing embedded name/email with FK to Organizer.
- Team: TeamID → LeaderName → LeaderEmail — resolved by replacing with LeaderUserID (FK → Users).
- Registration: RegistrationID → UserName → (Email, Phone) — resolved by using UserID FK.
- Payment: PaymentID → VerifierName → VerifierEmail — resolved by using VerifiedBy UserID FK.

3NF — Users (base table; role-specific columns removed)

UserID	Name	Email	Phone	PasswordHash
1	Muhammad Taha Farrukh	mtahafarrukh@gmail.com	03332758016	$2a$11$RZi...
2	Feras Malik	ferasmalik@ned.com	03331234567	$2a$11$qFc...
4	Saad Ali	saadali@ned.com	03332758017	$2a$11$zM1...
5	UsaidUllah Qureshi	usaid@ned.com	03121231231	$2a$11$Lpb...
6	Uzair Ahmed	uzair@ned.com	03332758018	$2a$11$ayQ...

3NF — Student, Organizer, Admin, Volunteer Sub-Tables

Each sub-entity shares the UserID primary key with the parent Users table (table-per-hierarchy pattern).

Student

UserID (PK = FK→Users)	RollNumber	Department
100	CS-24-100	Computer Science
101	CS-24-101	Computer Science
102	CS-24-102	Computer Science
103	CS-24-103	Computer Science
104	CS-24-104	Computer Science

Primary Key = UserID | Foreign Key: UserID → Users | RollNumber and Department depend only on UserID (student-specific attributes).

Organizer

UserID (PK = FK→Users)
4
295
296
297
298

Primary Key = UserID | Foreign Key: UserID → Users

Admin

UserID (PK = FK→Users)
5
7
325
326
327

Primary Key = UserID | Foreign Key: UserID → Users

Volunteer

UserID (PK = FK→Users)
6
310
311
312
313

3NF — Event Table (CreatedBy_Name/Email replaced with FK)

EventID	Name	Department	Location	StartDate	EndDate	CreatedBy (FK→Organizer)
1	TechFest 2026	CSIT	Department	2026-01-05	2026-01-06	4

3NF — Team Table (LeaderName/Email replaced with LeaderUserID FK)

TeamID	TeamName	LeaderUserID (FK→Users)	CompetitionID (FK→Competition)	(TeamName, CompetitionID) Candidate Key
1	Muhammad Taha Farrukh	1	2	—
2	Main Akela he kaafi hun	2	2	—

3NF — TeamMember Table

MemberID	TeamID (FK→Team)	Name	RollNumber	Department	Email
1	1	Muhammad Taha Farrukh	CT-24224	CSIT	mtahafarrukh@gmail.com
2	1	Feras Malik	CT-24247	CSIT	ferasmalik@ned.com
3	2	Feras Malik	CT-24247	CSIT	ferasmalik@ned.com
4	2	Muhammad Taha Farrukh	CT-24224	CSIT	mtahafarrukh@gmail.com

3NF — Registration Table (UserName → UserID FK)

RegID	CmpID (FK)	UserID (FK→Users)	TeamID (FK→Team)	Type	Status	RegisteredAt
2	2	1	1	Team	Approved	2026-04-10 22:22:40
3	2	2	2	Team	Approved	2026-04-10 22:53:58

3NF — Payment Table (VerifierName → VerifiedBy FK)

PayID	RegID (FK)	Screenshot	Status	VerifiedBy (FK→Users)	SubmittedAt	VerifiedAt
1	2	d4fb7e03-295c....jpeg	Approved	4	2026-04-10 22:23:00	2026-04-10 22:32:40
2	3	010f2187-046f....jpeg	Approved	6	2026-04-10 22:54:08	2026-04-10 22:54:36

3NF — Ticket Table

TicketID	RegID (FK→Registration)	QRCode (UUID)	UniqueCode	GeneratedAt
3	2	17b0d20a-c9ce-45d7-b99c-5fd07145b177	92C4E9BB	2026-04-10 22:32:41
4	3	31377175-d2a7-4159-bf72-c7246c64ad8d	198935D4	2026-04-10 22:54:36

9.6 Boyce-Codd Normal Form (BCNF)

A table is in BCNF if for every non-trivial functional dependency X → Y, X must be a superkey. BCNF is stricter than 3NF, it addresses anomalies from non-trivial FDs where the determinant is not a candidate key.

BCNF Violation Example

```
TeamMember_Old(TeamID, RollNumber, Name, Department)
-- FD1: (TeamID, RollNumber) → Name, Department   [candidate key]
-- FD2: RollNumber → Department                   ← BCNF VIOLATION
-- RollNumber is not a superkey, yet it determines Department
```

BCNF Fix

```
CREATE TABLE TeamMember (
  MemberID    SERIAL PRIMARY KEY,  -- MemberID is the sole superkey
  TeamID      INT NOT NULL REFERENCES Team(TeamID),
  Name        VARCHAR(150) NOT NULL,
  RollNumber  VARCHAR(50),
  Department  VARCHAR(100),
  Email       VARCHAR(255) NOT NULL
);
ALTER TABLE TeamMember
ADD CONSTRAINT uq_team_member_email UNIQUE (TeamID, Email);
```

BCNF Result: The final schema is in BCNF. Every functional dependency X → Y has X as a superkey (candidate key). All determinants are candidate keys. No update, insertion, or deletion anomalies exist. Note: TeamMember still stores RollNumber and Department together; if RollNumber → Department must be enforced at the DB level, Department should be removed from TeamMember and resolved via a JOIN with the Student table.

10. SQL Queries

10.1 DDL — Table Creation

```
-- 1. BASE USER TABLE (Super-entity)
CREATE TABLE Users (
  UserID  SERIAL PRIMARY KEY,
  Name    VARCHAR(150) NOT NULL,
  Email   VARCHAR(255) NOT NULL UNIQUE,
  Phone   VARCHAR(20)  NOT NULL
);

-- 2. STUDENT (Specialization of User)
CREATE TABLE Student (
  UserID      INT PRIMARY KEY REFERENCES Users(UserID) ON DELETE CASCADE,
  RollNumber  VARCHAR(50)  NOT NULL UNIQUE,
  Department  VARCHAR(100) NOT NULL
);

-- 3. ADMIN, ORGANIZER, VOLUNTEER
CREATE TABLE Admin     ( UserID INT PRIMARY KEY REFERENCES Users(UserID) ON
DELETE CASCADE );
CREATE TABLE Organizer ( UserID INT PRIMARY KEY REFERENCES Users(UserID) ON
DELETE CASCADE );
CREATE TABLE Volunteer ( UserID INT PRIMARY KEY REFERENCES Users(UserID) ON
DELETE CASCADE );

-- 4. ORGANIZER ROLE REQUEST
CREATE TABLE OrganizerRoleRequest (
  RequestID   SERIAL PRIMARY KEY,
  StudentID   INT NOT NULL REFERENCES Student(UserID) ON DELETE CASCADE,
  ApprovedBy  INT REFERENCES Admin(UserID) ON DELETE SET NULL,
  Status      VARCHAR(20) NOT NULL DEFAULT 'Pending'
              CHECK (Status IN ('Pending', 'Approved', 'Rejected')),
  RequestedAt TIMESTAMP NOT NULL DEFAULT NOW(),
  ReviewedAt  TIMESTAMP
);

-- 5. EVENT
CREATE TABLE Event (
  EventID    SERIAL PRIMARY KEY,
  Name       VARCHAR(200) NOT NULL,
  Department VARCHAR(100) NOT NULL,
  Location   VARCHAR(200) NOT NULL,
  StartDate  DATE NOT NULL,
  EndDate    DATE NOT NULL,
  CreatedBy  INT  NOT NULL REFERENCES Organizer(UserID) ON DELETE RESTRICT,
  CONSTRAINT chk_event_dates CHECK (EndDate >= StartDate)
);

-- 6. COMPETITION
CREATE TABLE Competition (
  CompetitionID SERIAL PRIMARY KEY,
  EventID       INT  NOT NULL REFERENCES Event(EventID) ON DELETE CASCADE,
  Name          VARCHAR(200) NOT NULL,
  Description   TEXT,
  Location      VARCHAR(200) NOT NULL,
  StartDate     DATE NOT NULL,
  EndDate       DATE NOT NULL,
  MaxTeamSize   INT  NOT NULL DEFAULT 1 CHECK (MaxTeamSize BETWEEN 1 AND 5),
  EntryFee      NUMERIC(10,2) NOT NULL DEFAULT 0.00,
```

```
  CONSTRAINT chk_comp_dates CHECK (EndDate >= StartDate)
);
```

```
-- 7. TEAM
CREATE TABLE Team (
  TeamID        SERIAL PRIMARY KEY,
  TeamName      VARCHAR(200) NOT NULL,
  LeaderUserID  INT NOT NULL REFERENCES Users(UserID) ON DELETE RESTRICT,
  CompetitionID INT NOT NULL REFERENCES Competition(CompetitionID) ON DELETE
CASCADE,
  CONSTRAINT uq_team_competition UNIQUE (TeamName, CompetitionID)
);

-- 8. TEAM MEMBER (BCNF fix: MemberID sole PK)
CREATE TABLE TeamMember (
  MemberID   SERIAL PRIMARY KEY,
  TeamID     INT NOT NULL REFERENCES Team(TeamID) ON DELETE CASCADE,
  Name       VARCHAR(150) NOT NULL,
  RollNumber VARCHAR(50),
  Department VARCHAR(100),
  Email      VARCHAR(255) NOT NULL,
  CONSTRAINT uq_team_member_email UNIQUE (TeamID, Email)
);

-- 9. REGISTRATION
CREATE TABLE Registration (
  RegistrationID SERIAL PRIMARY KEY,
  CompetitionID  INT NOT NULL REFERENCES Competition(CompetitionID) ON DELETE
RESTRICT,
  UserID         INT NOT NULL REFERENCES Users(UserID) ON DELETE RESTRICT,
  TeamID         INT REFERENCES Team(TeamID) ON DELETE SET NULL,
  Type           VARCHAR(20) NOT NULL CHECK (Type IN ('Individual','Team')),
  Status         VARCHAR(20) NOT NULL DEFAULT 'Pending'
                 CHECK (Status IN ('Pending','Approved','Rejected')),
  RegisteredAt   TIMESTAMP NOT NULL DEFAULT NOW(),
  CONSTRAINT uq_user_competition UNIQUE (UserID, CompetitionID)
);

-- 10. PAYMENT (1:1 with Registration)
CREATE TABLE Payment (
  PaymentID      SERIAL PRIMARY KEY,
  RegistrationID INT NOT NULL UNIQUE REFERENCES Registration(RegistrationID) ON
DELETE CASCADE,
  Screenshot     VARCHAR(500) NOT NULL,
  Status         VARCHAR(20) NOT NULL DEFAULT 'Pending'
                 CHECK (Status IN ('Pending','Approved','Rejected')),
  VerifiedBy     INT REFERENCES Users(UserID) ON DELETE SET NULL,
  SubmittedAt    TIMESTAMP NOT NULL DEFAULT NOW(),
  VerifiedAt     TIMESTAMP
);

-- 11. TICKET (Auto-generated on payment approval)
CREATE TABLE Ticket (
  TicketID       SERIAL PRIMARY KEY,
  RegistrationID INT  NOT NULL UNIQUE REFERENCES Registration(RegistrationID)
ON DELETE CASCADE,
  QRCode         TEXT NOT NULL UNIQUE DEFAULT gen_random_uuid()::TEXT,
  UniqueCode     VARCHAR(50) NOT NULL UNIQUE
                 DEFAULT UPPER(SUBSTRING(gen_random_uuid()::TEXT, 1, 8)),
  GeneratedAt    TIMESTAMP NOT NULL DEFAULT NOW()
);
```

10.2 DML — Data Insertion

```
-- Insert Users
INSERT INTO Users (Name, Email, Phone) VALUES
  ('Muhammad Taha Farrukh', 'mtahafarrukh@gmail.com', '03332758016'),
  ('Feras Malik',           'ferasmalik@ned.com',     '03331234567'),
  ('Saad Ali',              'saadali@ned.com',        '03332758017'),
  ('UsaidUllah Qureshi',    'usaid@ned.com',          '03121231231'),
  ('Uzair Ahmed',           'uzair@ned.com',          '03332758018');

INSERT INTO Student   (UserID, RollNumber, Department) VALUES
  (1, 'CT-24224', 'CSIT'), (2, 'CT-24247', 'CSIT');
INSERT INTO Organizer (UserID) VALUES (3);
INSERT INTO Admin     (UserID) VALUES (4);
INSERT INTO Volunteer (UserID) VALUES (5);

INSERT INTO Event (Name, Department, Location, StartDate, EndDate, CreatedBy)
VALUES
  ('TechFest 2026', 'CSIT', 'Main Auditorium', '2026-01-05', '2026-01-06', 3);

INSERT INTO Competition (EventID, Name, Description, Location, StartDate,
EndDate, MaxTeamSize, EntryFee) VALUES
  (1, 'Speed Programming', 'Competitive coding challenge', 'Lab 2', '2026-01-
05', '2026-01-05', 2, 1000.00);

-- Insert Teams
INSERT INTO Team (TeamName, LeaderUserID, CompetitionID) VALUES
  ('Muhammad Taha Farrukh', 1, 1),
  ('Main Akela he kaafi hun', 2, 1);

-- Insert Registrations
INSERT INTO Registration (CompetitionID, UserID, TeamID, Type, Status) VALUES
  (1, 1, 1, 'Team', 'Approved'),
  (1, 2, 2, 'Team', 'Approved');

-- Insert TeamMembers
INSERT INTO TeamMember (TeamID, Name, RollNumber, Department, Email) VALUES
  (1, 'Muhammad Taha Farrukh', 'CT-24224', 'CSIT', 'mtahafarrukh@gmail.com'),
  (1, 'Feras Malik',           'CT-24247', 'CSIT', 'ferasmalik@ned.com'),
  (2, 'Feras Malik',           'CT-24247', 'CSIT', 'ferasmalik@ned.com'),
  (2, 'Muhammad Taha Farrukh', 'CT-24247', 'CSIT', 'mtahafarrukh@gmail.com');

-- Insert Payments
INSERT INTO Payment (RegistrationID, Screenshot, Status, VerifiedBy,
SubmittedAt, VerifiedAt) VALUES
  (1, 'd4fb7e03-295c-4b7e-b0e0-f9aef6d12345.jpeg', 'Approved', 3,
   '2026-04-10 22:23:00', '2026-04-10 22:32:40'),
  (2, '010f2187-046f-4b3a-a1bc-d2e3f4a5b6c7.jpeg', 'Approved', 5,
   '2026-04-10 22:54:08', '2026-04-10 22:54:36');

-- Insert Tickets (auto-generated by trigger on Payment approval;
--                 shown here for completeness/manual reference)
INSERT INTO Ticket (RegistrationID, UniqueCode) VALUES
  (1, '92C4E9BB'),
  (2, '198935D4');
```

10.3 SELECT Queries

Query 1: All Events with Organizer Details

```
SELECT e.EventID, e.Name AS EventName, e.Department, e.StartDate, e.EndDate,
       u.Name AS OrganizerName, u.Email AS OrganizerEmail
FROM Event e
JOIN Organizer o ON e.CreatedBy = o.UserID
JOIN Users u ON o.UserID = u.UserID
ORDER BY e.StartDate;
```

Query 2: Registration Status with Payment & Ticket

```
SELECT r.RegistrationID, u.Name AS Participant, c.Name AS Competition,
       r.Type, r.Status AS RegStatus,
       p.Status AS PaymentStatus, t.UniqueCode AS TicketCode
FROM Registration r
JOIN Users u ON r.UserID = u.UserID
JOIN Competition c ON r.CompetitionID = c.CompetitionID
LEFT JOIN Payment p ON r.RegistrationID = p.RegistrationID
LEFT JOIN Ticket  t ON r.RegistrationID = t.RegistrationID
ORDER BY r.RegisteredAt DESC;
```

Query 3: Volunteer Count per Event

```
SELECT e.Name AS EventName, COUNT(es.UserID) AS VolunteerCount
FROM Event e
LEFT JOIN EventStaff es ON e.EventID = es.EventID
GROUP BY e.EventID, e.Name
ORDER BY VolunteerCount DESC;
```

Query 4: Teams with Member Count per Competition

```
SELECT c.Name AS Competition, t.TeamName,
       COUNT(tm.MemberID) AS CurrentMembers, c.MaxTeamSize
FROM Competition c
JOIN Team t ON c.CompetitionID = t.CompetitionID
LEFT JOIN TeamMember tm ON t.TeamID = tm.TeamID
GROUP BY c.CompetitionID, c.Name, t.TeamID, t.TeamName, c.MaxTeamSize
ORDER BY c.Name, t.TeamName;
```

Query 5: Pending Payments Awaiting Verification

```
SELECT p.PaymentID, u.Name AS Participant, c.Name AS Competition,
       p.SubmittedAt, p.Screenshot
FROM Payment p
JOIN Registration r ON p.RegistrationID = r.RegistrationID
JOIN Users u ON r.UserID = u.UserID
JOIN Competition c ON r.CompetitionID = c.CompetitionID
WHERE p.Status = 'Pending'
ORDER BY p.SubmittedAt ASC;
```

Query 6: Students Without Any Registration

```
SELECT u.UserID, u.Name, u.Email, s.RollNumber, s.Department
FROM Student s
JOIN Users u ON s.UserID = u.UserID
WHERE NOT EXISTS (
  SELECT 1 FROM Registration r WHERE r.UserID = s.UserID
);
```

10.4 UPDATE and DELETE Queries

```
-- UPDATE: Approve a payment
UPDATE Payment SET Status = 'Approved', VerifiedBy = 3, VerifiedAt = NOW()
WHERE PaymentID = 1;

-- UPDATE: Reject a registration
UPDATE Registration SET Status = 'Rejected' WHERE RegistrationID = 2;

-- DELETE: Remove a team member (cascade-safe)
DELETE FROM TeamMember WHERE MemberID = 3;

-- DELETE: Cancel a competition (cascades to teams, registrations, payments,
tickets)
DELETE FROM Competition WHERE CompetitionID = 1;
```

10.5 Views, Triggers, and Stored Procedures

Trigger 1: Auto-generate Ticket on Payment Approval

```
CREATE OR REPLACE FUNCTION auto_generate_ticket()
RETURNS TRIGGER AS $$
BEGIN
  IF NEW.Status = 'Approved' AND OLD.Status <> 'Approved' THEN
    INSERT INTO Ticket (RegistrationID)
    VALUES (NEW.RegistrationID)
    ON CONFLICT (RegistrationID) DO NOTHING;
  END IF;
  RETURN NEW;
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER trg_auto_ticket
AFTER UPDATE OF Status ON Payment
FOR EACH ROW EXECUTE FUNCTION auto_generate_ticket();
```

Trigger 2: Enforce MaxTeamSize

```
CREATE OR REPLACE FUNCTION check_team_size()
RETURNS TRIGGER AS $$
DECLARE current_size INT; max_size INT;
BEGIN
  SELECT COUNT(*) INTO current_size FROM TeamMember WHERE TeamID = NEW.TeamID;
  SELECT c.MaxTeamSize INTO max_size
  FROM Team t JOIN Competition c ON t.CompetitionID = c.CompetitionID
  WHERE t.TeamID = NEW.TeamID;
  IF current_size >= max_size THEN
    RAISE EXCEPTION 'Team size limit exceeded. Max allowed: %', max_size;
  END IF;
  RETURN NEW;
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER trg_team_size_limit
BEFORE INSERT ON TeamMember
FOR EACH ROW EXECUTE FUNCTION check_team_size();
```

Trigger 3: Payment Verified Only by Organizer or Volunteer

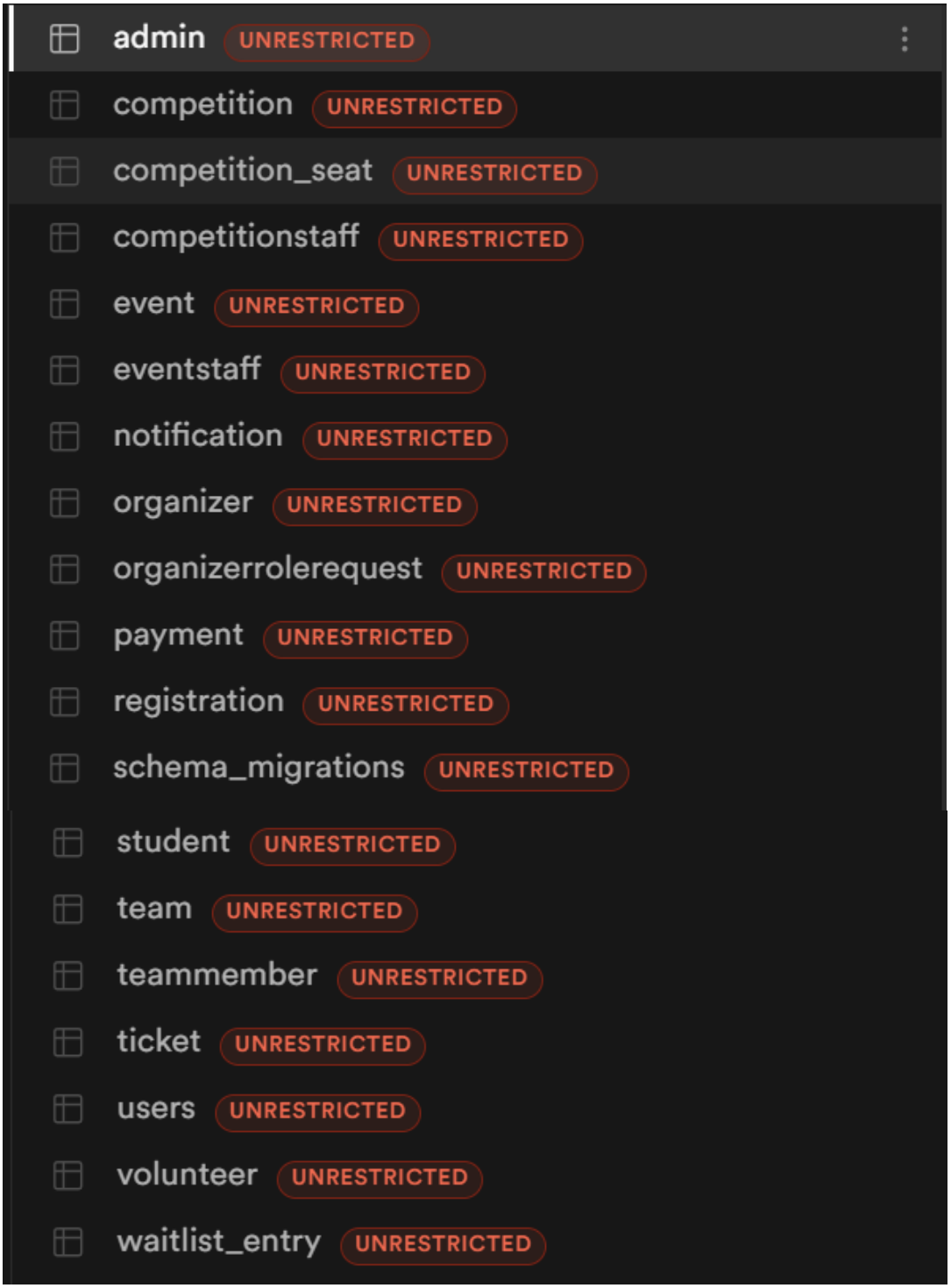
```
CREATE OR REPLACE FUNCTION check_payment_verifier()
RETURNS TRIGGER AS $$
BEGIN
  IF NEW.VerifiedBy IS NOT NULL THEN
    IF NOT EXISTS (
      SELECT 1 FROM Organizer WHERE UserID = NEW.VerifiedBy
      UNION
      SELECT 1 FROM Volunteer WHERE UserID = NEW.VerifiedBy
    ) THEN
      RAISE EXCEPTION 'Payment can only be verified by an Organizer or
Volunteer';
    END IF;
  END IF;
  RETURN NEW;
END; $$ LANGUAGE plpgsql;

CREATE TRIGGER trg_payment_verifier
BEFORE INSERT OR UPDATE ON Payment
FOR EACH ROW EXECUTE FUNCTION check_payment_verifier();
```

11. Screenshots / Query Outputs

The following screenshots demonstrate the database schema and query execution in Supabase (PostgreSQL).

11.1 Database Schema in Supabase



11.2 Query 1 Output — Events with Organizer Details

```sql
SELECT e.EventID,
       e.Name AS EventName,
       e.Department,
       u.Name AS OrganizerName,
       u.Email AS OrganizerEmail
FROM Event e
JOIN Organizer o ON e.CreatedBy = o.UserID
JOIN Users u ON o.UserID = u.UserID;
```

Results Explain Chart Export Source Primary Database Role postgres

eventid	eventname	department	organizername	organizeremail
3	Test Event	CSIT	Saad Ali	saadali@ned.com
1	TechFest 2026	CSIT	Saad Ali	saadali@ned.com
2	hackathon 26	CSIT	khan	alikhan123@gmail.com
4	Test Event 01	CSIT	Test Organizer	testorganizer@cloud.neduet.edu.pk

11.3 Query 2 Output — Registration Status with Payment & Ticket

Fetch Event Organizer Details

```sql
SELECT r.RegistrationID,
       u.Name,
       p.Status AS PaymentStatus,
       t.UniqueCode
FROM Registration r
JOIN Users u ON r.UserID = u.UserID
LEFT JOIN Payment p ON r.RegistrationID = p.RegistrationID
LEFT JOIN Ticket t ON r.RegistrationID = t.RegistrationID;
```

Results Explain Chart Export Source

registrationid	name	paymentstatus	uniquecode
2	Muhammad Taha Farrukh	Approved	92C4E9BB
3	Feras Malik	Approved	198935D4
4	Feras Malik	Approved	D68F1FFE
5	Muhammad Taha Farrukh	NULL	NULL
6	Feras Malik	Approved	92CFA2AB
7	Test Admin	Rejected	NULL
8	Test Student	Approved	8D93E3F9

11.4 Trigger Demonstration — Auto Ticket Generation

paymentid	registrationid i...	screenshot varchar	status varchar
1	2	/uploads/payments/d4fb7e03-295c-418d-	Approved
2	3	/uploads/payments/010f2187-046f-4dce-8	Approved
3	4	/uploads/payments/e5faae56-f490-4139-8	Approved
4	6	/uploads/payments/91676c34-5f99-426b-	Approved
5	7	/uploads/payments/a700bdf8-42a1-4527-9	Rejected
6	8	/uploads/payments/6aa176e9-bc97-42a4-	Approved
7	9	/uploads/payments/f4793737-11f3-4139-8	Approved

ticketid i..	registrationid i...	qrcode text	uniquecode varchar
3	2	17b0d20a-c9ce-45d7-b99c-5fd07145b177	92C4E9BB
4	3	31377175-d2a7-4159-bf72-c7246c64ad8d	198935D4
5	4	6883fdfc-c4c8-4aeb-bca1-31e1845374a1	D68F1FFE
6	6	8b9331a9-1fcf-435d-9eec-b5851dc52dcd	92CFA2AB
7	9	cf27620c-4421-459f-aada-b863e9819b43	DF9A32DE
8	8	160d959c-9a92-4f70-bd32-41286e290d2(	8D93E3F9

11.5 Additional Outputs

Team Member Count per Competition

Fetch Event Organizer Details

```sql
SELECT c.Name AS Competition,
       t.TeamName,
       COUNT(tm.MemberID) AS TotalMembers
FROM Competition c
JOIN Team t ON c.CompetitionID = t.CompetitionID
LEFT JOIN TeamMember tm ON t.TeamID = tm.TeamID
GROUP BY c.Name, t.TeamName
ORDER BY c.Name;
```

Results Explain Chart Export

competition	teamname	totalmembers
Blind Code	Amigos	2
Speed Programming	Main Akela he kaafi hun	2
Speed Programming	Muhammad Taha Farrukh	2
Speed Programming	NEDians	2
Speed Programming	NEDians 2	2

Students Without Registration

Fetch Event Organizer Details

```sql
SELECT u.UserID,
       u.Name,
       s.RollNumber
FROM Student s
JOIN Users u ON s.UserID = u.UserID
WHERE NOT EXISTS (
    SELECT 1
    FROM Registration r
    WHERE r.UserID = s.UserID
);
```

Results Explain Chart Export

userid	name	rollnumber
251	Safoora Qamar	ME-24-251
106	Zubair Ahmed	CS-24-106
285	Amina Waqar	TE-23-285
120	Junaid Aslam	CS-24-120
264	Rizwan Haider	IE-24-264
209	Iffat Arshad	EE-23-209

Total Revenue per Event

Fetch Event Organizer Details

```sql
SELECT e.Name AS EventName,
       SUM(c.EntryFee) AS TotalRevenue
FROM Event e
JOIN Competition c ON e.EventID = c.EventID
GROUP BY e.Name;
```

Results Explain Chart Export

eventname	totalrevenue
Test Event 01	1000.00
TechFest 2026	4999.99

Approved Tickets List

Fetch Event Organizer Details

```sql
SELECT u.Name,
       t.UniqueCode
FROM Ticket t
JOIN Registration r ON t.RegistrationID = r.RegistrationID
JOIN Users u ON r.UserID = u.UserID
JOIN Competition c ON r.CompetitionID = c.CompetitionID;
```

Results Explain Chart Export

name	competition	uniquecode
Muhammad Taha Farrukh	Speed Programming	92C4E9BB
Feras Malik	Speed Programming	198935D4
Feras Malik	AI Paradox	D68F1FFE
Feras Malik	Blind Code	92CFA2AB
Test Student	Speed Programming	8D93E3F9
Test Student	Speed Programming	DF9A32DE

12. Group Member Contributions

Roll No.	Member Name	Sections Responsible	Tools Used
CT-24224	Muhammad Taha Farrukh	Backend & Database Integration (Sec 10, Triggers), bug fixing.	ASP.NET, Supabase, Documentation
CT-24221	Muhammad Saad	Anomalies, Normalization UNF → BCNF (Sec 8, 9)	MS Word, Excel
CT-24227	UsaidUllah Qureshi	Frontend Enhancement	Razor Pages, Bootstrap
CT-24247	Muhammad Feras Malik	Database Design, ER/EER, Database Implementation	draw.io, pgAdmin, Supabase

13. Limitations

13.1 Technical Limitations

- The system currently supports a single university domain. Multi-campus support would require additional tenant-based schema changes.
- Supabase's free tier has storage and row limits that may be reached during large-scale university-wide deployment.
- QR code generation uses UUID-based values; a dedicated QR library integration is not yet implemented in the database layer.
- The schema does not yet include full-text search indexes for event/competition discovery by keyword.

13.2 Design Limitations

- The current schema includes a WAITLIST_ENTRY table with priority-number ordering, but automatic promotion logic (moving a waitlisted registration to confirmed when a seat opens) is not yet implemented as a database trigger.
- Payment supports only screenshot-based verification. Integration with a payment gateway (JazzCash, EasyPaisa) is outside the current scope.
- The Ticket entity generates a static QR code at approval time. Dynamic QR scanning for attendance tracking is not implemented.

13.3 Time & Resource Constraints

The project was completed within a single semester with a team of four students. This limited the scope to core normalization, schema design, and sample queries. A full front-end integration with ASP.NET Core and live Supabase deployment was designed but not fully implemented within the project timeline.

14. Future Enhancements

14.1 Short-Term Enhancements

- Implement an automatic waitlist-promotion trigger so that when a confirmed registration is cancelled, the highest-priority waitlisted registration is automatically upgraded and the student is notified.
- Add email/SMS notification triggers to inform students of payment approval and ticket generation.
- Build a reporting module with pre-defined analytical queries accessible to admins.
- Add full-text search on Event and Competition names using PostgreSQL tsvector indexes.

14.2 Long-Term Enhancements

- Integrate a payment gateway (JazzCash / EasyPaisa) to replace manual screenshot verification.
- Develop a dynamic QR scanning mobile app for real-time attendance tracking at events.
- Extend the schema to support multi-campus deployments with tenant-based row-level security.
- Build a public event portal front-end using ASP.NET Core / Razor Pages connected to Supabase.

15. Conclusion

The Smart Campus Event Ticketing & Real-Time Event Portal System database has been successfully designed and normalized to Boyce-Codd Normal Form (BCNF). The schema consists of 18 tables covering all required entities, relationships, and business rules specified in the project overview, drawn from a thorough analysis of the real-world campus event management problem at NED University.

The normalization process began with a single large unnormalized (UNF) relation and proceeded step-by-step through 1NF (elimination of repeating groups), 2NF (removal of partial dependencies), 3NF (elimination of transitive dependencies by separating the User super-entity into sub-tables), and finally we tried BCNF (resolving the RollNumber → Department FD in TeamMember). Each step was demonstrated with concrete sample data drawn from the actual system.

Key achievements of this design include: full normalization to BCNF eliminating all insertion, deletion, and update anomalies; correct EER specialization with overlapping and partial participation; three many-to-many relationships modeled via junction tables; ten or more business rules enforced at the database layer through triggers and CHECK constraints; a complete DDL and DML SQL script ready for one-click deployment on Supabase; and comprehensive sample analytical queries covering joins, aggregations, subqueries, and NOT EXISTS patterns.

The complete source code and SQL scripts are available at: github.com/EventHub-NED

Declaration of Academic Integrity

We hereby declare that this report is our own original work and has not been submitted for assessment in any other unit or course. All sources have been appropriately acknowledged.

— End of Report —

CT-261 | Spring Semester 2026 | NED University of Engineering & Technology